



Customer Churn Prediction - Telecom

Internship Project by Mohsin Khan

A comprehensive predictive model to identify at-risk customers.

1: Data Generation

Below we run the provided code to generate the mock dataset for 2,000 customers._

```
In [ ]: import pandas as pd
import numpy as np

# (Rest of imports if you need seaborn/matplotlib later)
import seaborn as sns
import matplotlib.pyplot as plt

In [ ]: # Generate mock customer data
np.random.seed(42)
num_customers = 2000

data = {
    'CustomerID': [f'CUST{1000+i}' for i in range(num_customers)],
    'Gender': np.random.choice(['Male', 'Female'], num_customers, p=[0.5, 0.5]),
    'SeniorCitizen': np.random.choice([0, 1], num_customers, p=[0.84, 0.16]),
    'Partner': np.random.choice(['Yes', 'No'], num_customers, p=[0.48, 0.52]),
    'Dependents': np.random.choice(['Yes', 'No'], num_customers, p=[0.3, 0.7]),
    'Tenure': np.random.randint(1, 73, num_customers), # Months
    'PhoneService': np.random.choice(['Yes', 'No'], num_customers, p=[0.9, 0.1]),
    'MultipleLines': np.random.choice(
        ['Yes', 'No', 'No phone service'], num_customers, p=[0.42, 0.48, 0.1]
    ),
    'InternetService': np.random.choice(
        ['DSL', 'Fiber optic', 'No'], num_customers, p=[0.34, 0.44, 0.22]
    ),
    'OnlineSecurity': np.random.choice(
        ['Yes', 'No', 'No internet service'], num_customers, p=[0.28, 0.50, 0.22]
    ),
    'OnlineBackup': np.random.choice(
        ['Yes', 'No', 'No internet service'], num_customers, p=[0.34, 0.44, 0.22]
    ),
    'DeviceProtection': np.random.choice(
        ['Yes', 'No', 'No internet service'], num_customers, p=[0.34, 0.44, 0.22]
    ),
    'TechSupport': np.random.choice(
        ['Yes', 'No', 'No internet service'], num_customers, p=[0.29, 0.49, 0.22]
    ),
    'StreamingTV': np.random.choice(
        ['Yes', 'No', 'No internet service'], num_customers, p=[0.38, 0.40, 0.22]
    ),
}
```

```

    'StreamingMovies': np.random.choice(
        ['Yes', 'No', 'No internet service'], num_customers, p=[0.39, 0.39, 0.22]
    ),
    'Contract': np.random.choice(
        ['Month-to-month', 'One year', 'Two year'], num_customers, p=[0.55, 0.24, 0.21]
    ),
    'PaperlessBilling': np.random.choice(['Yes', 'No'], num_customers, p=[0.59, 0.41]),
    'PaymentMethod': np.random.choice(
        ['Electronic check', 'Mailed check', 'Bank transfer (automatic)', 'Credit card'],
        num_customers, p=[0.34, 0.23, 0.22, 0.21]
    ),
    'MonthlyCharges': np.random.normal(loc=65, scale=30, size=num_customers).clip(18, 120)
}

df_customers = pd.DataFrame(data)

# Generate TotalCharges based on Tenure and MonthlyCharges with some noise
df_customers['TotalCharges'] = (
    df_customers['Tenure']
    * df_customers['MonthlyCharges']
    * np.random.uniform(0.95, 1.05, num_customers)
).round(2)

# Make some TotalCharges empty for realism (e.g., new customers with 0 tenure)
df_customers.loc[df_customers['Tenure'] == 1, 'TotalCharges'] = df_customers['MonthlyCharges']

# **Fix:** select 1% of those with Tenure < 3, then set TotalCharges to NaN
tenure_lt_3_idx = df_customers[df_customers['Tenure'] < 3].index
random_idx = np.random.choice(tenure_lt_3_idx, size=int(num_customers * 0.01), replace=False)
df_customers.loc[random_idx, 'TotalCharges'] = np.nan

# Simulate Churn (more likely for month-to-month, higher charges, Lower tenure)
churn_probability = (
    0.1
    + 0.15 * (df_customers['Contract'] == 'Month-to-month')
    + 0.1 * (df_customers['InternetService'] == 'Fiber optic')
    + 0.001 * (df_customers['MonthlyCharges'] - 65)
    - 0.002 * (df_customers['Tenure'] - 36)
    + 0.1 * (df_customers['OnlineSecurity'] == 'No')
    + 0.1 * (df_customers['TechSupport'] == 'No')
)
churn_probability = np.clip(churn_probability, 0.01, 0.99)

df_customers['Churn'] = np.random.binomial(1, churn_probability, num_customers).astype(int)
df_customers['Churn'] = df_customers['Churn'].replace({'1': 'Yes', '0': 'No'})

# Replace 'No phone service' and 'No internet service' for consistency
for col in ['MultipleLines']:
    df_customers[col] = df_customers.apply(
        lambda row: 'No' if row['PhoneService'] == 'No' else row[col], axis=1
    )

for col in ['OnlineSecurity', 'OnlineBackup', 'DeviceProtection', 'TechSupport', 'StreamingMovies']:
    df_customers[col] = df_customers.apply(
        lambda row: 'No' if row['InternetService'] == 'No' else row[col], axis=1
    )

```

```
# Save to CSV
df_customers.to_csv('telecom_churn_mock_data.csv', index=False)
print("Mock telecom churn data generated: telecom_churn_mock_data.csv")
print(df_customers.head())
print(f"\nChurn distribution:\n{df_customers['Churn'].value_counts(normalize=True)}")
```

2. Display Top 5 Rows Of Dataset

```
In [ ]: df_customers.head()
```

3. Display Last 5 Rows Of Dataset

```
In [ ]: df_customers.tail()
```

4. Find shape of our dataset (number of rows,number of columns)

```
In [ ]: df_customers.shape
```

```
In [ ]: print("number of Rows" ,df_customers.shape[0])
print("number of columns" ,df_customers.shape[1])
```

5. get information about our data set like total number of rows,total number of columns,datatypes of each column and memory requirments

```
In [ ]: df_customers.info()
```

6. Check Null Values In the dataset

```
In [ ]: df_customers.isnull().sum()
```

```
In [ ]: # Dropping the Nulls
df = df_customers.dropna(subset=['TotalCharges'])
df.isnull().sum()
```

7. Get overall Statistics About The Dataset

```
In [ ]: df.describe(include="all")
```

7. Checking Irrelevant features

```
In [ ]: df.columns
```

8. Dropping Columns CustomerID ,PhoneService

Reasons

CustomerID isJust an identifier, adds zero predictive value. DROP IT.

PhoneService is redundant because if PhoneService = 'No', then MultipleLines is 'No phone service', so we already get that info. DROP ONE of the two. Preferably drop PhoneService.

```
In [ ]: df = df.drop(['CustomerID', 'PhoneService'], axis=1)
```

```
In [ ]: df.head()
```

9.Encoding Categorical Data

```
In [ ]: #df1 is data after encoding  
df1 = pd.get_dummies(df, drop_first=True)
```

```
In [ ]: df1.head()
```

```
In [ ]: # Convert all boolean columns to integers (0/1)  
df1 = df1.astype(int)  
df1.head()
```

10. check that Data is BAlanced Or not

```
In [ ]: df1['Churn_Yes'].value_counts()
```

```
In [ ]: import seaborn as sns  
import matplotlib.pyplot as plt
```

```
# Custom color palette: No = green, Yes = red
sns.countplot(data=df, x='Churn', palette={'No': 'green', 'Yes': 'red'})

plt.title("Churn Count")
plt.xlabel("Churn")
plt.ylabel("Number of Customers")
plt.show()
```

Review

its clearly shows that our data is imbalanced

solution

we are going to use SMOTE for handing Imbalanced Data

```
In [ ]: #Step 1: Import Required Libraries for SMOTE and Model Training
# For splitting the dataset
from sklearn.model_selection import train_test_split

# For oversampling
from imblearn.over_sampling import SMOTE

# For model training and evaluation
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix
```

```
In [ ]: # Step 2: Separate Features and Target
X = df1.drop('Churn_Yes', axis=1) # All columns except churn
y = df1['Churn_Yes']             # Target column: churn
```

```
In [ ]: #Step 3: Split Data Into Train and Test Sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

```
In [ ]: #Step 4: Apply SMOTE on Training Data ONLY
smote = SMOTE(random_state=42) # Create SMOTE object

X_train_smote, y_train_smote = smote.fit_resample(X_train, y_train) # Apply it
```

```
In [ ]: import numpy as np

unique, counts = np.unique(y_train_smote, return_counts=True)
for label, count in zip(unique, counts):
    print(f"Class {label}: {count} samples")
```

11. Applying Logisticregression model

```

In [ ]: # 1. Instantiate the model
model = LogisticRegression(max_iter=1000, random_state=42)

# 2. Fit on the SMOTE-balanced training set
model.fit(X_train_smote, y_train_smote)

In [ ]: # Predict churn on the untouched test data
y_pred = model.predict(X_test)

In [ ]: # 1. Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:\n", cm)

# 2. Classification Report (precision, recall, f1-score)
print("\nClassification Report:\n", classification_report(y_test, y_pred))

```

Why This Happened (Even After SMOTE):

SMOTE helped balance the training set, but:

Logistic Regression is a linear model — may not capture complex churn patterns

There might be non-linear relationships, interactions, or irrelevant features.

12. Random Forest Classifier

```

In [ ]: from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import confusion_matrix, classification_report

# Initialize and train
rf_model = RandomForestClassifier(random_state=42)
rf_model.fit(X_train_smote, y_train_smote)

# Predict
y_pred_rf = rf_model.predict(X_test)

# Evaluate
print("🔍 Random Forest Results:")
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_rf))
print("\nClassification Report:\n", classification_report(y_test, y_pred_rf))

```

13. Gradient Boosting Classifier

```

In [ ]: from sklearn.ensemble import GradientBoostingClassifier

# Initialize and train
gb_model = GradientBoostingClassifier(random_state=42)

```

```
gb_model.fit(X_train_smote, y_train_smote)

# Predict
y_pred_gb = gb_model.predict(X_test)

# Evaluate
print("🔍 Gradient Boosting Results:")
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_gb))
print("\nClassification Report:\n", classification_report(y_test, y_pred_gb))
```

14. Support Vector Machine (SVM)

```
In [ ]: from sklearn.svm import SVC

# Initialize and train
svc_model = SVC(probability=True, random_state=42)
svc_model.fit(X_train_smote, y_train_smote)

# Predict
y_pred_svc = svc_model.predict(X_test)

# Evaluate
print("🔍 SVM Results:")
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_svc))
print("\nClassification Report:\n", classification_report(y_test, y_pred_svc))
```

15. Try XGBoost

```
In [ ]: from xgboost import XGBClassifier

xgb_model = XGBClassifier(random_state=42, use_label_encoder=False, eval_metric='logit')
xgb_model.fit(X_train_smote, y_train_smote)

y_pred_xgb = xgb_model.predict(X_test)

print("🔍 XGBoost Results:")
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_xgb))
print("\nClassification Report:\n", classification_report(y_test, y_pred_xgb))
```

ROC Curve + AUC Score

For Logistic Regression model's performance

```
In [ ]: from sklearn.metrics import roc_curve, auc
import matplotlib.pyplot as plt

# Get predicted probabilities for the positive class (churn = 1)
y_probs = model.predict_proba(X_test)[:, 1]
```

```

# Calculate ROC curve values
fpr, tpr, thresholds = roc_curve(y_test, y_probs)

# Calculate AUC score
roc_auc = auc(fpr, tpr)
print(f"ROC AUC Score: {roc_auc:.4f}")

# Plot the ROC curve
plt.figure(figsize=(8,6))
plt.plot(fpr, tpr, color='blue', label=f'ROC curve (AUC = {roc_auc:.4f})')
plt.plot([0, 1], [0, 1], color='gray', linestyle='--', label='Random Guess')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve for Logistic Regression')
plt.legend(loc='lower right')
plt.grid(True)
plt.show()

```

Random Forest model

```

In [ ]: from sklearn.metrics import roc_curve, auc
import matplotlib.pyplot as plt

# Get predicted probabilities for positive class (churn=1)
y_probs_rf = rf_model.predict_proba(X_test)[: , 1]

# Calculate ROC curve values
fpr_rf, tpr_rf, thresholds_rf = roc_curve(y_test, y_probs_rf)

# Calculate AUC score
roc_auc_rf = auc(fpr_rf, tpr_rf)
print(f"Random Forest ROC AUC Score: {roc_auc_rf:.4f}")

# Plot the ROC curve
plt.figure(figsize=(8,6))
plt.plot(fpr_rf, tpr_rf, color='green', label=f'ROC curve (AUC = {roc_auc_rf:.4f})')
plt.plot([0, 1], [0, 1], color='gray', linestyle='--', label='Random Guess')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve for Random Forest Classifier')
plt.legend(loc='lower right')
plt.grid(True)
plt.show()

```

Gradient Boosting Classifier

```

In [ ]: from sklearn.metrics import roc_curve, auc
import matplotlib.pyplot as plt

# Get predicted probabilities for positive class (churn=1)

```



```

y_probs_gb = gb_model.predict_proba(X_test)[: , 1]

# Calculate ROC curve values
fpr_gb, tpr_gb, thresholds_gb = roc_curve(y_test, y_probs_gb)

# Calculate AUC score
roc_auc_gb = auc(fpr_gb, tpr_gb)
print(f"Gradient Boosting ROC AUC Score: {roc_auc_gb:.4f}")

# Plot the ROC curve
plt.figure(figsize=(8,6))
plt.plot(fpr_gb, tpr_gb, color='blue', label=f'ROC curve (AUC = {roc_auc_gb:.4f})')
plt.plot([0, 1], [0, 1], color='gray', linestyle='--', label='Random Guess')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve for Gradient Boosting Classifier')
plt.legend(loc='lower right')
plt.grid(True)
plt.show()

```

Support Vector Machine (SVM)

```

In [ ]: from sklearn.metrics import roc_curve, auc
import matplotlib.pyplot as plt

# Get predicted probabilities for positive class (churn=1)
y_probs_svc = svc_model.predict_proba(X_test)[: , 1]

# Calculate ROC curve values
fpr_svc, tpr_svc, thresholds_svc = roc_curve(y_test, y_probs_svc)

# Calculate AUC score
roc_auc_svc = auc(fpr_svc, tpr_svc)
print(f"SVM ROC AUC Score: {roc_auc_svc:.4f}")

# Plot the ROC curve
plt.figure(figsize=(8,6))
plt.plot(fpr_svc, tpr_svc, color='purple', label=f'ROC curve (AUC = {roc_auc_svc:.4f})')
plt.plot([0, 1], [0, 1], color='gray', linestyle='--', label='Random Guess')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve for SVM Classifier')
plt.legend(loc='lower right')
plt.grid(True)
plt.show()

```

XGBoost

```

In [ ]: from sklearn.metrics import roc_curve, auc
import matplotlib.pyplot as plt

```

```

# Predict probabilities for positive class (churn=1)
y_probs_xgb = xgb_model.predict_proba(X_test)[: , 1]

# Calculate ROC curve
fpr_xgb, tpr_xgb, _ = roc_curve(y_test, y_probs_xgb)

# Calculate AUC
roc_auc_xgb = auc(fpr_xgb, tpr_xgb)
print(f"XGBoost ROC AUC Score: {roc_auc_xgb:.4f}")

# Plot ROC curve
plt.figure(figsize=(8,6))
plt.plot(fpr_xgb, tpr_xgb, color='orange', label=f'ROC curve (AUC = {roc_auc_xgb:.4f})')
plt.plot([0, 1], [0, 1], color='gray', linestyle='--', label='Random Guess')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve for XGBoost Classifier')
plt.legend(loc='lower right')
plt.grid(True)
plt.show()

```

Model Comparison Gather all Our scores and ROC-AUC

```

In [ ]: from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score,

models = {
    'Logistic Regression': model,
    'Random Forest': rf_model,
    'Gradient Boosting': gb_model,
    'SVM': svc_model
}

results = []

for name, mdl in models.items():
    y_pred = mdl.predict(X_test)
    y_proba = mdl.predict_proba(X_test)[: , 1] # For ROC AUC

    results.append({
        'Model': name,
        'Accuracy': accuracy_score(y_test, y_pred),
        'Precision': precision_score(y_test, y_pred),
        'Recall': recall_score(y_test, y_pred),
        'F1 Score': f1_score(y_test, y_pred),
        'ROC AUC': roc_auc_score(y_test, y_proba)
    })

import pandas as pd
results_df = pd.DataFrame(results)
print(results_df.sort_values(by='ROC AUC', ascending=False))

```

Hyperparameter tuning with GridSearchCV for Logistic Regression and Random Forest

```
In [ ]: from sklearn.model_selection import GridSearchCV

# Logistic Regression hyperparameter grid
param_grid_lr = {
    'C': [0.01, 0.1, 1, 10],
    'penalty': ['l2'], # L1 might cause convergence issues, but you can try it later
    'solver': ['lbfgs', 'saga']
}

grid_lr = GridSearchCV(LogisticRegression(max_iter=1000, random_state=42), param_grid_lr)
grid_lr.fit(X_train_smote, y_train_smote)

print("Best Logistic Regression Params:", grid_lr.best_params_)
print("Best Logistic Regression ROC AUC:", grid_lr.best_score_)
```

```
In [ ]: # Random Forest hyperparameter grid
param_grid_rf = {
    'n_estimators': [100, 200],
    'max_depth': [5, 10, None],
    'min_samples_split': [2, 5],
    'min_samples_leaf': [1, 2]
}

grid_rf = GridSearchCV(RandomForestClassifier(random_state=42), param_grid_rf, cv=5)
grid_rf.fit(X_train_smote, y_train_smote)

print("Best Random Forest Params:", grid_rf.best_params_)
print("Best Random Forest ROC AUC:", grid_rf.best_score_)
```

This means our model's now wayyy better at distinguishing churners from non-churners.

```
In [ ]: # Retrain best RF model
best_rf = RandomForestClassifier(
    max_depth=10,
    min_samples_leaf=1,
    min_samples_split=2,
    n_estimators=100,
    random_state=42
)

best_rf.fit(X_train_smote, y_train_smote)

# Predict on test data
y_pred_best_rf = best_rf.predict(X_test)
```

```

y_pred_prob_rf = best_rf.predict_proba(X_test)[: , 1]

# Metrics
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score,

print("🔍 Tuned Random Forest Test Results:")
print("Accuracy:", accuracy_score(y_test, y_pred_best_rf))
print("Precision:", precision_score(y_test, y_pred_best_rf))
print("Recall:", recall_score(y_test, y_pred_best_rf))
print("F1 Score:", f1_score(y_test, y_pred_best_rf))
print("ROC AUC:", roc_auc_score(y_test, y_pred_prob_rf))
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred_best_rf))
print("\nClassification Report:\n", classification_report(y_test, y_pred_best_rf))

```

Summary & Key Takeaways

- The tuned Random Forest model achieved a test ROC AUC of **0.569** and accuracy of **63%**.
- It performs better at predicting the majority class (non-churn), but struggles with minority class (churn), as seen from lower recall.
- Top features (from feature importance) include: `tenure` , `MonthlyCharges` , and `Contract` type.
- Future improvements could involve:
 - Trying more advanced models like XGBoost or LightGBM.
 - Further feature engineering or dimensionality reduction.
 - Collecting more data or using cost-sensitive learning techniques.

```

In [ ]: import joblib
        joblib.dump(svc_model, "tuned_random_forest.pkl")

```

```

In [ ]:

```